Architecture and Trade-offs

In-memory account ledger · Nitin Maurya · 1 November 2026 · code at tag `v1.2` · figures measured with `docs/bench/`

The decisions everything else follows from

Decision	Buys	Costs in production
Store facts, derive state: two append-only lists; balances and statuses computed, never stored	Nothing drifts; any past figure can be rebuilt	Every query is a full scan (section 1)
Two dates per entry; a day's balance is everything value-dated on or before it	A correction is one new line	Closed days change after the fact (section 2)
Money as whole fils in a `long`, tied to its currency	Exact sums; no mixing AED and BHD	FX has to be an explicit step
Refusals are recorded results with a reason	"Why was this declined?" always has an answer	The log grows with refusals too

1. Append-only at scale

CPU breaks first, not memory. After every request the ledger re-checks every closed day of every account for a missing fee, scanning every entry each time, so a run grows like *requests* × *accounts* × *entries*: 3.4 s at 100× volume, 96 s at 300×, on only about 6,000 records. Authorization status is also rebuilt from the log per request, so card latency degrades early.

Unbounded state: the log and entries grow forever, holds never expire, and the fee re-check always starts from Day 1.

Cheapest fix: a running total per account per value date, updated on append and rebuildable from the log. The re-check then touches one account from one date onwards. A period close later bounds growth over time.

2. Value-dated entries in production

Day 5 closed at −230.00 with three fees and ended at 465.00. Statements already shown are wrong, the declined Auth-B can't be undone, and past fees and interest must be re-run as new entries. CBUAE consumer protection requires refunding charges caused by the bank's own error immediately, which is why my refunds carry each fee's own date. Every back-dated entry also needs an audit trail.

Control before go-live: maker-checker. An entry dated before its booking day needs a reason code and a second approver. My ledger lacks this; it is the test I left failing on purpose.

3. Authorization lifecycle

Ending	My ledger	Real world	What I'd mandate
Declined for funds	DECLINED, no hold (Auth-B)	Card declined at the till	Decline code and customer notice
Refused, structural	Also shows as DECLINED	Blocked card, purchase abroad	Distinct reason codes

Duplicate ID	Resend refused, no second hold	Network retry	Return the original answer
Never settled	Hold blocks money forever	Unused deposit, cancelled order	Expiry as a new event
Cancelled first	Can't be expressed	Merchant void	A void that releases the hold
Settlement doesn't fit	Refused, hold stays	Tip over the hold	Tolerance, then exceptions queue

A settlement with no authorization (Auth-Z) moves nothing from the account, but the scheme still expects payment, so production routes it to a suspense account.

4. What I cut and why

Cut	Risk it defers
Concurrency	Two authorizations could spend the same money
Persistence	A restart loses everything
Maker-checker, reason codes	One person rewrites closed days; every reversal refunds fees
Debit limits	Any debit is accepted, however large
Date checks	A settlement or reversal dated before its original is accepted
Hold expiry and voids	Money locked by holds that never settle
Calendar, FX, double-entry	No business days, foreign spending or GL reconciliation

I cut these to keep every number checkable by hand and spend the effort on what the brief tests.